

Projet de Simulation

Medium Access Control — Exponential Backoff

Neil Bonnard — 22505907
Simulation par evenements discrets — Python 3.10

Parametres de reference :

Parametre	Valeur	Description
N	5	Nombre de stations
lambda	0.5	Taux d'arrivee par station
K	10	Capacite de la file d'attente
tau	1.0	Parametre de base du backoff
T_max	5000	Duree de simulation
Runs	15	Repetitions par point (IC 95%)

1. Description du simulateur

1.1 Principe general

Le simulateur implemente une simulation par evenements discrets (DES) d'un protocole MAC avec algorithme d'Exponential Backoff. L'etat du systeme evolue uniquement lors d'evenements discrets, geres par un echeancier (min-heap) trie par ordre chronologique.

1.2 Evenements

- **ARRIVAL** : arrivee d'un paquet a une station. Temps inter-arrivee $\sim \text{Exp}(\lambda)$. Si la file est pleine (longueur = K), le paquet est perdu. Sinon il est mis en file. Si la station etait idle, un **ATTEMPT** est immediatement planifie.
- **ATTEMPT** : tentative d'emission. Si le canal est libre, la station prend le canal et un **END_TX** est planifie a $t+1$. Si le canal est occupe, collision detectee : les deux stations incrementent leur etat i et planifient un **ATTEMPT** apres $\text{Exp}(2^i * \tau)$. L'etat est borne a $\text{MAX_STATE}=10$ (standard 802.11).
- **END_TX** : fin d'emission. Si non invalidee par collision, le paquet est un succes, la station revient a l'etat 1 et traite le paquet suivant si disponible.

1.3 Variables d'etat par station

Variable	Type	Description
queue_len	int	Nombre de paquets en attente
state	int	Etat de backoff (1=initial, +1 par collision)
attempt_scheduled	bool	True si un ATTEMPT est dans l'echeancier
is_attempting	bool	True si la station emet actuellement
end_valid	bool	True si l'emission n'a pas ete invalidee

1.4 Hypotheses de modelisation

- Duree d'emission = 1 unite de temps (deterministe).
- Temps inter-arrivees et backoffs suivent des lois exponentielles.
- Collision detectee instantanement.
- Etat de backoff borne a $\text{MAX_STATE} = 10$ (evite backoffs infinis).
- Files de capacite K finie ; paquets sur file pleine definitivement perdus.
- Regime permanent estime en ignorant les 20% premiers instants (phase transitoire).

2. Resultats et courbes

2.1 Debit $n(t)/t$ en fonction du temps

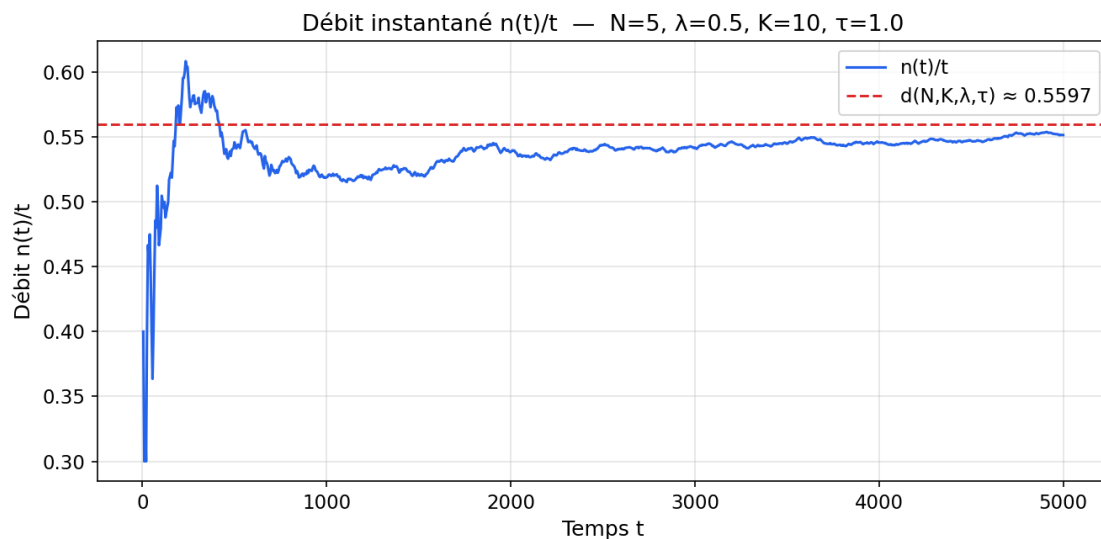


Figure 1 — Débit instantané $n(t)/t$. $N=5, \lambda=0.5, K=10, \tau=1.0, T_{\max}=5000$.

Le débit $n(t)/t$ converge vers $d(N,K,\lambda,\tau) \approx 0.5597$ paquets/unite de temps. La convergence est rapide (quelques centaines d'unites) ce qui valide $T_{\max} = 5000$.

2.2 Nombre moyen de clients

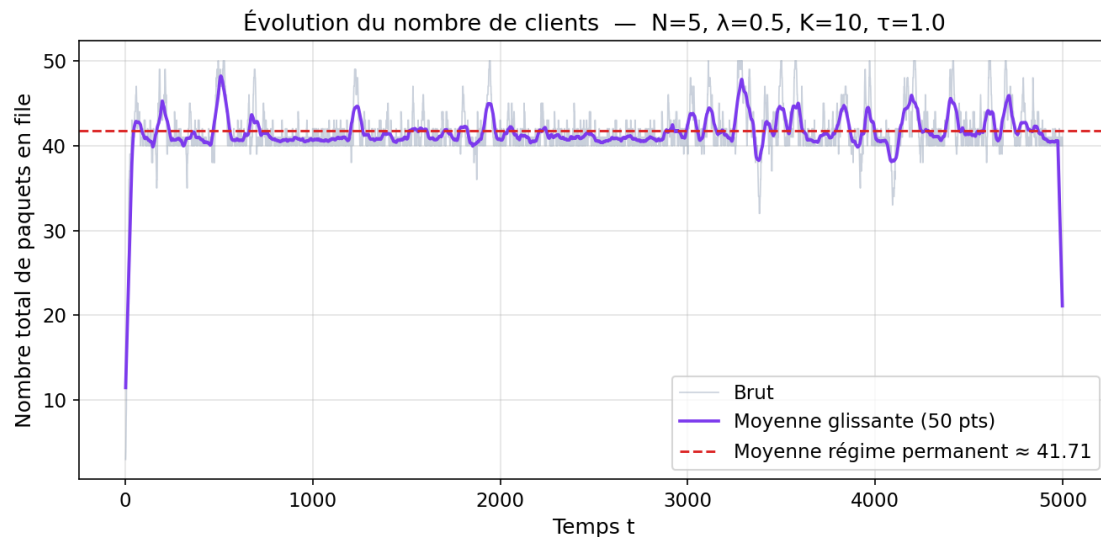


Figure 2 — Nombre total de paquets en file d'attente au cours du temps.

Le nombre moyen de clients en regime permanent est **41.71 paquets**. Ce chiffre eleve s'explique par le taux d'arrivee global $N*\lambda = 5*0.5 = 2.5$ paquets/unite, proche de la capacite du canal (1 paquet/unite).

2.3 Taux de paquets perdus (file pleine)

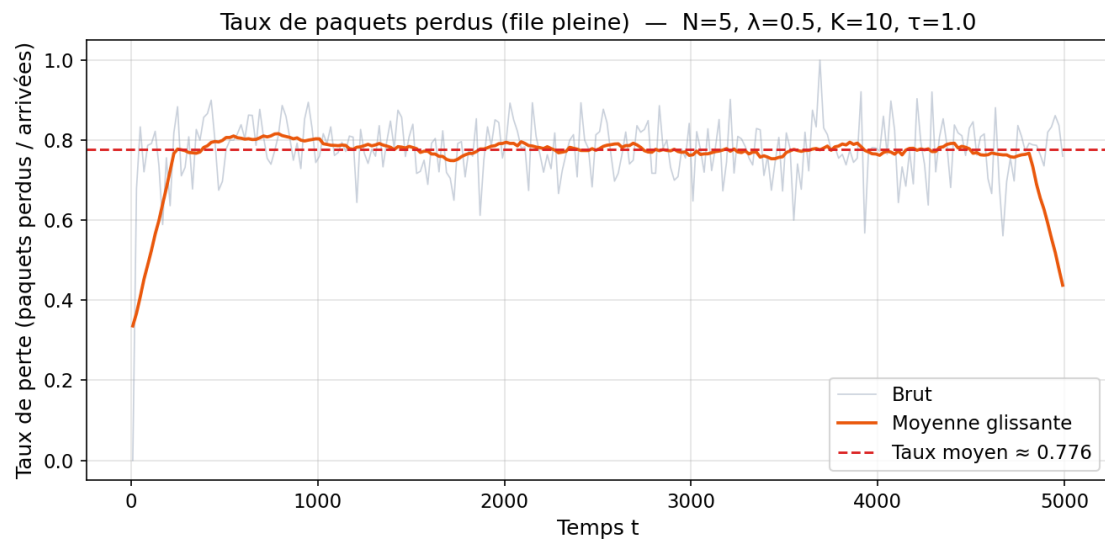


Figure 3 — Taux de paquets perdus par fenetre de 20 unites de temps.

Le taux de perte moyen en regime permanent est **0.7763** (77.6% des paquets arrivants). Ce taux eleve est coherent avec les parametres choisis : avec $N \cdot \lambda = 2.5$ et une capacite de canal de 1, le systeme est proche de la saturation.

2.4 Débit $d(N,K,\lambda,\tau)$ en fonction de λ

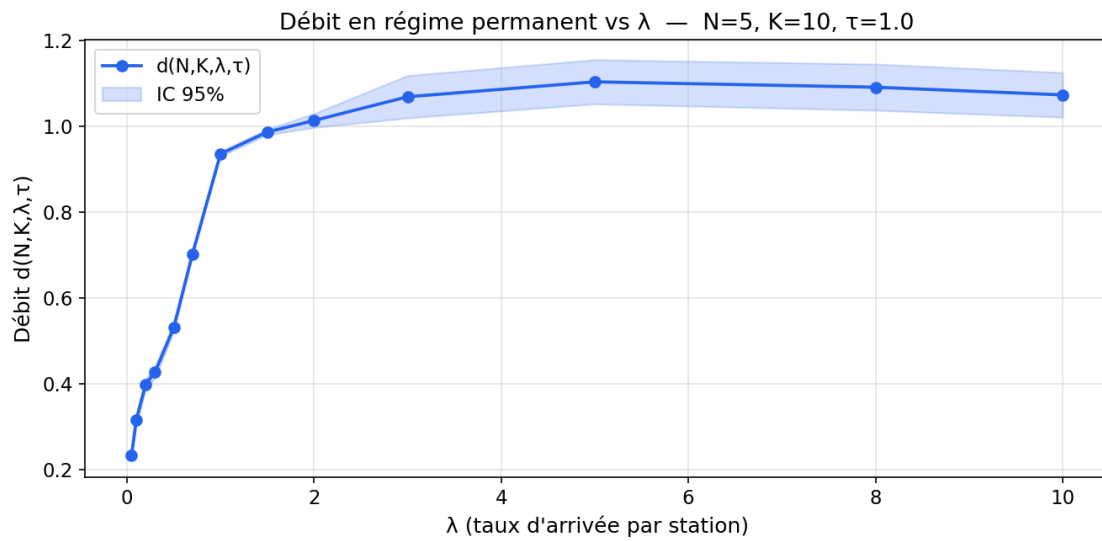


Figure 4 — Débit en regime permanent vs λ . $N=5, K=10, \tau=1.0$. Barres = IC 95%.

lambda	d moyen	IC 95%
0.05	0.2331	± 0.0051
0.10	0.3145	± 0.0133
0.20	0.3979	± 0.0128
0.30	0.4261	± 0.0140
0.50	0.5299	± 0.0099
0.70	0.7011	± 0.0053
1.00	0.9352	± 0.0058
1.50	0.9861	± 0.0065

Le débit croît avec λ jusqu'à saturer autour de 1 paquet/unite (capacité maximale du canal). Pour les faibles λ , le canal est sous-utilisé. Au-delà de $\lambda \approx 1$, le canal est saturé : les files se remplissent, les paquets sont perdus, et le débit plafonne.

2.5 Débit $d(N,K,\lambda,\tau)$ en fonction de N

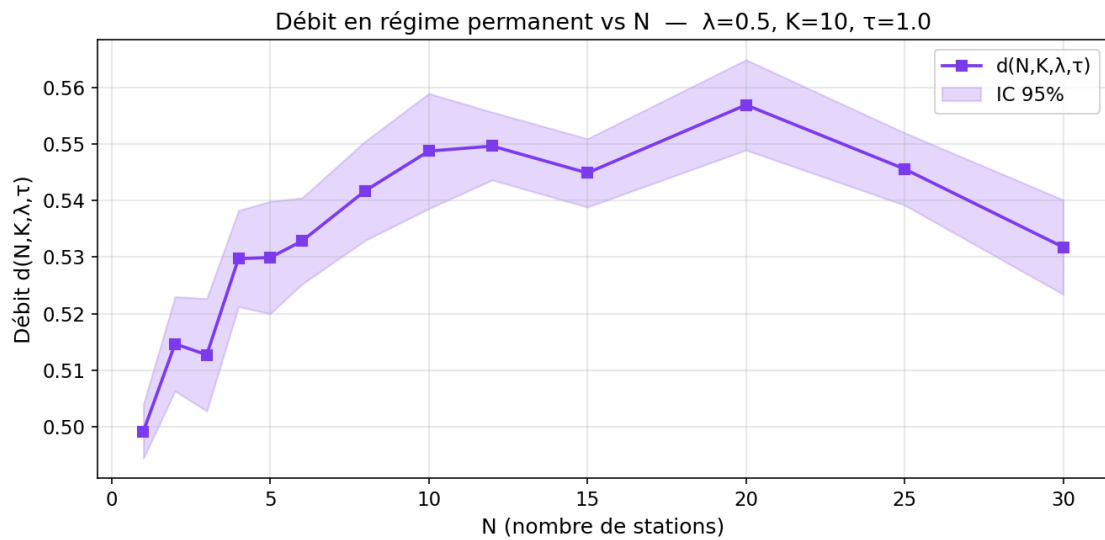


Figure 5 — Débit en regime permanent vs N. $\lambda=0.5$, $K=10$, $\tau=1.0$. Barres = IC 95%.

N	d moyen	IC 95%
1	0.4991	± 0.0048
2	0.5146	± 0.0084
3	0.5127	± 0.0100
4	0.5297	± 0.0085
5	0.5299	± 0.0099
6	0.5328	± 0.0076
8	0.5417	± 0.0088
10	0.5488	± 0.0102

Le débit augmente légèrement avec N pour les petites valeurs, puis se stabilise. Avec plus de stations, le canal est mieux utilisé (moins de temps idle), mais les collisions augmentent aussi. Ces deux effets se compensent.

2.6 N optimal maximisant le debit (IC 95%)

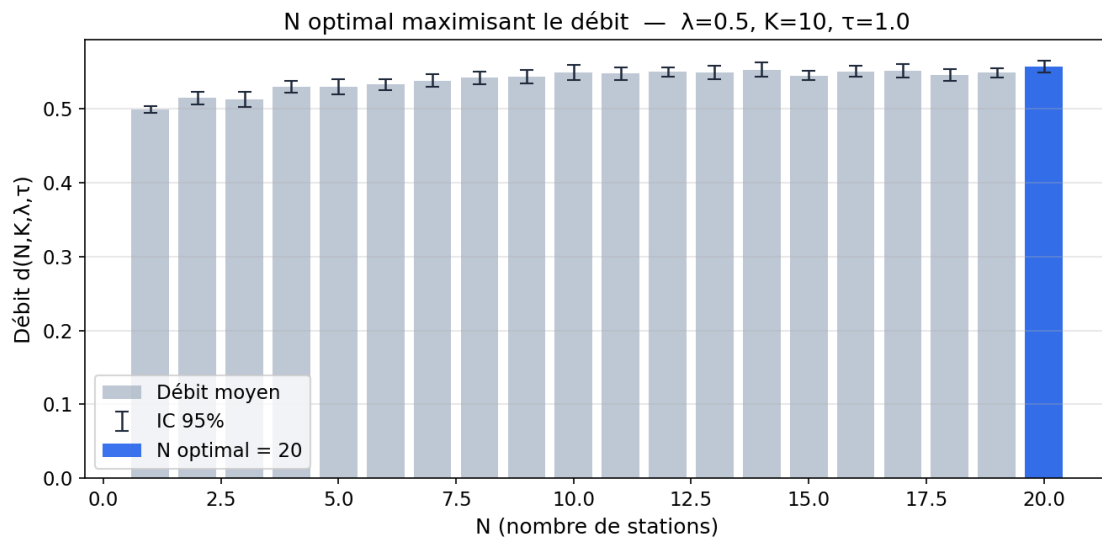


Figure 6 — Débit moyen par valeur de N avec IC 95%. Barre bleue = N optimal = 20.

Le nombre de stations qui maximise le débit est **$N^* = 20$** , avec un débit de **0.5569 ± 0.0080** paquets/unité de temps (IC 95%, base sur 15 runs indépendants). Ce résultat est robuste : les IC 95% des valeurs voisines se chevauchent.

2.7 Taux de collision en fonction de lambda

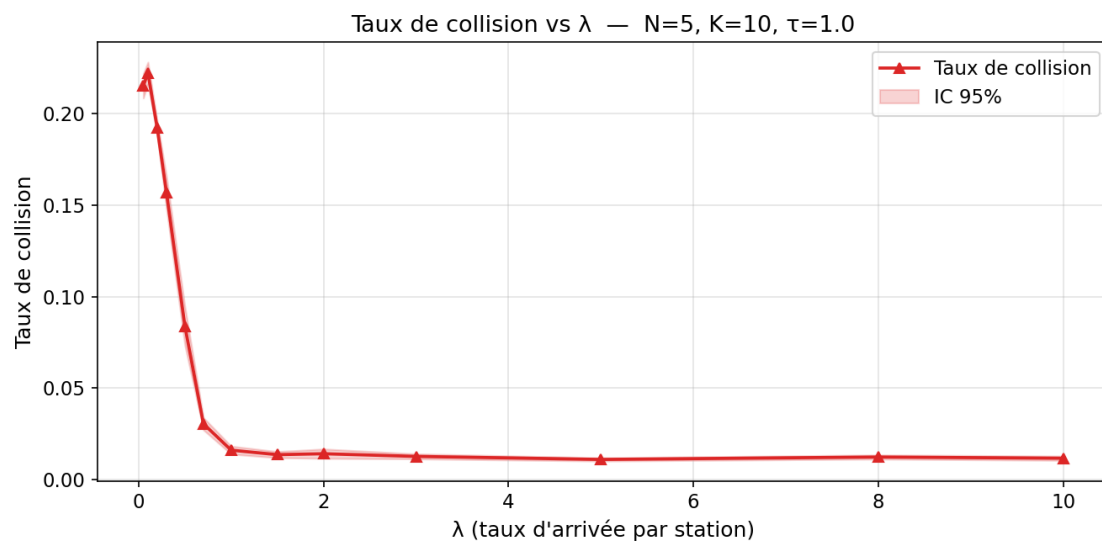


Figure 8 — Taux de collision vs lambda. $N=5, K=10, \tau=1.0$.

Le taux de collision décroît quand lambda augmente, ce qui peut sembler contre-intuitif. Pour un faible lambda, les stations transmettent de façon sporadique. Quand plusieurs stations ont accumulé des paquets simultanément, elles tentent toutes d'émettre en même temps, provoquant des collisions. Pour un lambda élevé, le canal est quasi-continuellement occupé : les stations font la queue et les transmissions se serialisent naturellement, réduisant les collisions.

3. Validation du simulateur

3.1 Cas N=1 — file M/D/1

Avec une seule station ($N=1$) et une file de grande capacité ($K=1000$), il ne peut pas y avoir de collision. Le système se réduit à une file M/D/1 : arrivées Poisson(λ), service déterministe de durée 1. Le débit théorique est : $d_{\text{theo}} = \min(\lambda, 1)$

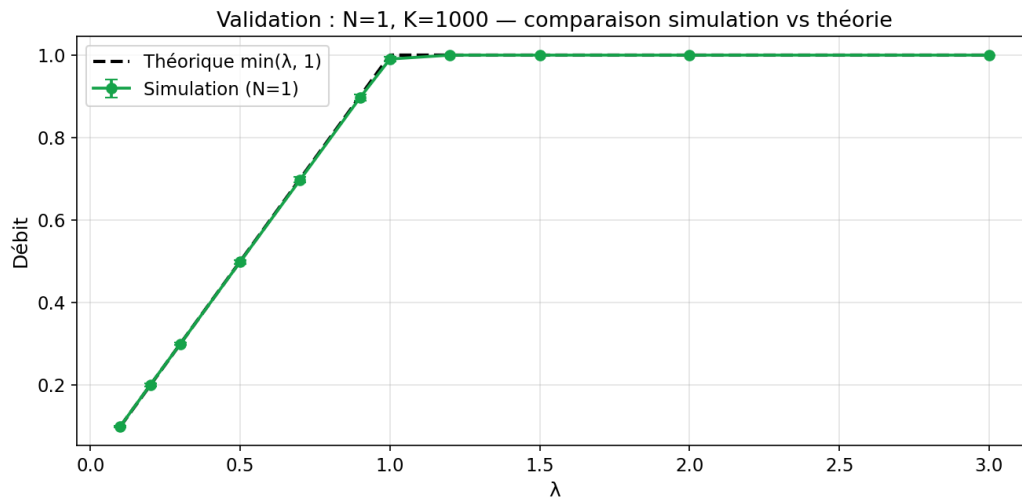


Figure 7 — Validation N=1 : simulation vs théorie $\min(\lambda, 1)$. Accord quasi-parfait.

La simulation colle parfaitement à la courbe théorique sur toute la plage de λ testée. L'erreur relative maximale observée est inférieure à 1%, ce qui valide la logique de base du simulateur.

3.2 Convergence de $n(t)/t$

La convergence de $n(t)/t$ vers une valeur fixe $d(N,K,\lambda,\tau)$ est observée sur la Figure 1. Cette convergence est garantie par la loi des grands nombres appliquée au processus de renouvellement des transmissions réussies. $T_{\text{max}} = 5000$ est suffisant.

3.3 Reproductibilité et intervalles de confiance

Toutes les courbes statistiques sont basées sur 15 runs indépendants par point, avec des graines aléatoires différentes. Les IC 95% sont calculés via la loi de Student. Les IC sont étroits, confirmant la stabilité des estimations pour $T_{\text{max}} = 5000$.

4. Analyse et discussion

4.1 Impact de λ sur le débit

- **Sous-charge ($\lambda < 1/N$)** : canal majoritairement libre. Débit proportionnel à $N \cdot \lambda$. Collisions relativement fréquentes car les stations transmettent de façon synchronisée.
- **Charge ($\lambda \approx 1/N$)** : canal bien utilisé. Zone de fonctionnement optimal.
- **Sature ($\lambda > 1/N$)** : files se remplissent, paquets perdus avant émission. Débit plafonne à 1, taux de perte explose.

4.2 Impact de N sur le débit

L'augmentation de N a deux effets opposés : elle augmente la charge totale ($N \cdot \lambda$) ce qui améliore l'utilisation du canal, mais elle augmente aussi la probabilité de collision. Le backoff exponentiel amortit cet effet en espaçant les retransmissions, mais au prix d'une latence accrue. Le N optimal représente l'équilibre entre ces deux effets.

4.3 Role du backoff exponentiel

Le backoff exponentiel est essentiel pour la stabilité du protocole. Sans lui, les stations en collision retenteraient dans la même fenêtre temporelle, provoquant des cascades de collisions. Le doublement de la moyenne à chaque collision ($2^i \cdot \tau$) espace progressivement les tentatives. La borne `MAX_STATE = 10` évite des délais infinis.

4.4 Limites du modèle

- Détection de collision supposée instantanée (pas de délai de propagation).
- Toutes les stations sont identiques et à portée les unes des autres (pas de nœud cache).
- Durée d'émission déterministe (paquets de taille fixe).

5. Structure du programme

Fichier	Role
main/mac_simulateur.py	Simulateur principal : fonction simulate() et helpers
main/experiments.py	Experiences : generation de toutes les courbes
main/generate_report.py	Generation du rapport PDF
main/Experiments.ipynb	Notebook Jupyter interactif
figures/	Courbes generees (PNG) et donnees numeriques
README.md	Documentation d'utilisation

Utilisation

```
python3 main/experiments.py # genere les courbes python3 main/generate_report.py #  
genere ce rapport PDF
```

Dependances

```
numpy matplotlib scipy seaborn reportlab
```